Getting trades in: file imports and the hand-entry route

How a trade enters the journal from a file, and by hand. The third way, the Alpaca accounts, has its own document: ACCOUNTS.md.

Written from `journal/src/journal/importer.js`, `journal/src/routes/journal.js` and the Import tab of `journal/public/journal.html`. Every behaviour marked **tested** was confirmed by running the real importer on a scratch database.

The three ways in

Way	Code	Trade id	Can carry a stop?	Status
File import (Import tab)	`importer.js`	random (uuid)	no	always `closed`
Alpaca account (Trades tab panel)	`addons/accounts.js`	from the account and the first fill: stable	no	`open` or `closed`
By hand (API only)	`POST /api/journal/trades`	random (uuid)	yes	`closed` when an exit price is given

None of the three sets a stop on its own, and the page has no field for one, so every R-based number on the page is empty (KNOWN-ISSUES.md, D-J9).

---

1. File import

On the page

The **Import** tab: **Choose CSV** (reads a `.csv` or `.txt` into the text box) or paste into the box, an optional **account label**, then **Import**. **Clear** empties the box. **Re-eval Conditions** is explained in ANALYSIS.md.

The page sends `POST /api/journal/import` with `{ csv, account }`. The answer is `{ ok, imported, total, trades, bridged }`, or `{ ok: false, error }` with status 400. The page then reloads every trade.

Step 1: delimiter and format

1. Blank lines are dropped. Fewer than two lines → `CSV too short`.
2. **Delimiter:** the first line is counted; more `;` than `,` means `;`.
3. **Format:** the header cells are lower-cased and stripped of everything but letters (`Date/Time` → `datetime`, `Order ID` → `orderid`). The first rule that matches wins:

Order	Format	Rule on the cleaned headers
1	DXtrade orders history	delimiter is `;` and has `symbol`, `side`, `event`
2	Trade The Pool orders history	has `datetime`, `orderid`, `event`
3	TradingView fills export	has `symbol`, `side`, `qty`
4	Trade The Pool trades (older)	has `action`, `shares`
5	TradingView paper-trading log	exactly two headers, `time` and `text`

Anything else is refused: `Unknown CSV format. Supported: TradingView fills, TTP, DXtrade, TradingView paper log.`

Step 2: each format becomes a list of fills

A fill is `{ ticker, ts, side, shares, price, commission }`. `side` is one of `buy`, `sell`, `sell_short`, `buy_cover`.

Times. Every format's clock time is read as **New York time** and turned into a real instant (the code finds that date's New York offset by asking what New York's hour is at 12:00 UTC). Accepted shapes: `YYYY-MM-DD HH:MM[:SS]`, `MM/DD/YYYY HH:MM[:SS]`, and for DXtrade `DD/MM/YYYY HH:MM:SS AM/PM`. A row whose time does not parse is skipped.

Format	Columns read	Side rule	Commission	Notes
TradingView fills	`Symbol`, `Side`, `Qty`, `Price`, `Commission`, `Date` (header names matched exactly, case-insensitive)	the `Side` cell lower-cased: `buy` or `sell`	from the file	the ticker **keeps its exchange prefix** (`NASDAQ:AAPL`)
TTP trades	`Symbol` (or `Sym`), `Action`, `Side`, `Shares`, `Price`, `Commission`, `Date`, `Time`	`Action=open` → `buy`, or `sell_short` when `Side=short`; anything else → `sell`, or `buy_cover` when `Side=short`	from the file	
TTP orders history	sent to the TTP trades parser	as above	as above	see the problem below
DXtrade	found by part of the header name: `date`, `symbol`, `side`, `type`, `price`, `stop`, `quantity`, `event`	only rows with `Event=Filled`. `Type=Market` is an **entry** (`Buy` → `buy`, `Sell` → `sell_short`). `Type=Stop loss` or `Take profit` is an **exit** (`Buy` → `buy_cover`, `Sell` → `sell`). Every other type is ignored	0	an exit's price is its stop price when one is given; `EX:` prefixes are removed from the ticker
TradingView paper log	`Time`, `Text`. Lines like `Order 3149586192 for symbol NYSE:HPE has been executed at price 49.15 for 695.1 units` are the fills; lines like `Call to place market order to buy 300 units of symbol NYSE:HPE` give the side	each executed line takes the side of a queued "call" line for the same symbol, preferring one with the same share count	0	**throws on any file with a "call" line** (below)

Step 3: fills become round trips (`matchTrades`)

This is the part a rebuild must not copy. Read it exactly:

1. All fills are sorted by time and keyed by **ticker only**.

2. An **entry** fill (`buy` , `buy_long` , `sell_short`) opens a position for that ticker **if none is open**. Direction is `Long` for a buy and `Short` for a short sale. The position takes **that one fill's** price, shares and commission.
3. **Any further entry fill while a position is open is ignored.**
4. The **first exit** fill (any other side) **closes the whole position at that fill's price**, for the shares of the opening fill. Commission = the opening fill's + this fill's.
5. An exit fill with no open position is **ignored**.
6. Positions still open at the end of the file are **dropped**. They are not stored, and the answer does not mention them.

Each completed trade is stored with:

Field	Value
`id`	a new random uuid
`date`	the entry's New York date
`entry_time` , `exit_time`	New York clock, `HH:MM`
`gross_pnl`	(exit − entry) × shares, reversed for a short
`commission` , `net_pnl`	as above; net = gross − commission
`pct_move`	(exit − entry) ÷ entry × 100, **not** reversed for a short
`duration_ms`	exit instant − entry instant
`source`	always `import` , whatever the format
`account`	the label typed on the page, or empty
`status`	`closed`

The rows are written with `INSERT OR IGNORE` . Because every id is new, nothing is ever ignored: **importing the same file twice stores every trade twice**.

What the import does next

For the newly stored trades the route starts, in the background and without waiting, the bar fetch and technical compute for each one, 300 ms apart (ANALYSIS.md). It also offers each to the grading copy, which skips them all because an imported trade has no setup yet (ANALYSIS.md).

Tested behaviour

The real importer was run on small files built for each case:

Case	What should be stored	What is stored
TradingView fills: buy 100 @ 10, buy 100 @ 11, sell 100 @ 12, sell 100 @ 13, $1 each	one long of 200 shares, net **$396**	one long of **100** shares, 10 → 12, net **$198**, ticker `NASDAQ:AAPL`
TradingView fills: sell 50 @ 20, buy 50 @ 19 (a short)	one short, +$50	**nothing** (the sell is an exit with nothing open; the buy opens a long that never closes and is dropped)
TradingView paper log with buy and sell	one long	**the import fails**: `Cannot read properties of undefined (reading 'toUpperCase')`. The “call” pattern has three capture groups and the code reads groups 3 and 4
TTP orders history (`Date/Time` , `Order ID` , … `Event`)	one long	**nothing**. That file goes to the TTP trades parser, which looks for `Date` , `Time` and `Action` columns: the times do not parse, so every row is skipped
DXtrade: market buy, then a market sell to close	one long	**nothing**. The market sell is read as a short entry, ignored because a position is open, and the long never sees an exit
The same TradingView file imported twice	one trade	**two** trades

The 84 journal tests in the code repo all pass, but none of them covers `importer.js`. That is why these went unnoticed. They are recorded as D-J1 to D-J7 in KNOWN-ISSUES.md.

---

2. By hand: `POST /api/journal/trades`

No form on the page calls it.

Field	Rule
`date`, `ticker`, `direction`, `shares`, `entry_price`, `entry_time`	required, else 400
`direction`	`Long`, or anything else is treated as a short in the P&L
`exit_price`, `exit_time`	optional; with an exit price the trade is `closed`, without it `open`
`sl`, `tp`, `commission`, `setup_id`, `notes`, `account`	optional
`context`	optional market-context object, stored as the trade's context snapshot (DATA-MODEL.md)

Computed: gross P&L, net = gross − commission, `pct_move` **signed by direction**, `duration_ms` from the two clock times on the entry date. `shares` is cut to a whole number. `source` = `manual`. A closed trade is offered to the grading copy at once.

---

3. Editing and deleting

`PATCH /api/journal/trades/:id` accepts `exit_price`, `exit_time`, `sl`, `tp`, `setup_id`, `notes` and `status`:

- gross, net (with the stored commission), `pct_move` (signed) and duration are recomputed;
- status becomes the one sent, else `closed` when there is an exit price, else it stays;
- sending `exit_price`, `exit_time`, `sl` or `tp` marks the bars for re-fetching (`technical_computed = 0`); a setup or a note alone does not;
- a closed trade is offered to the grading copy, which **skips it if a card already exists**, so changing the setup later does not move the card;
- when a setup is set on a closed trade that already has a technical snapshot, that setup's conditions are evaluated at once.

On the page only the setup can be changed: the dropdown on each trade card. Commission cannot be edited anywhere.

`DELETE /api/journal/trades/:id` removes the trade, its two snapshots, its conditions and its grading card. On the page the 🗑 button asks for a second tap (no browser dialog) and reports a failure instead of hiding it.

What a rebuild must do instead

1. **Pair fills by running position**, per account and ticker, exactly as the Alpaca path already does (ACCOUNTS.md): volume-weighted entry and exit, a reversal split into two trades, open positions kept as open trades.
2. **One parser per format, each with a sample file and a test**, including shorts, scale-ins, scale-outs and partial closes.
3. **A stable id from the file’s own facts** (account, ticker, first fill time and price, or the broker’s order id), so a second import updates instead of duplicating.
4. **One ticker spelling**: strip exchange prefixes everywhere.
5. **Report what was not used**: unmatched fills, open positions, skipped rows.
6. **A trade has an entry date and an exit date**, not one date and two clock times.